

System Design Brief

Timothy Clarke
University of Dundee
United Kingdom
2712139@dundee.ac.uk

1. Introduction

The system proposed in this document is to allow registered customers to search and book conference rooms in locations around the country. Conference rooms have a maximum capacity attribute. Locations may have one or more conference rooms. The booking price is determined by the room's base price attribute, plus an adjustment based on the temperature forecast for the day (see Table 10 of Appendix A). Bookings are made for a full day.

Section 2 proposes a technology agnostic solution. The technology stack is discussed in Section 3, with two technical solutions compared for each component, and one chosen with justification given.

We have not been tasked with implementing log-analysis, and for this assessment the assumption is made that one will not be required.

2. Architectural Design

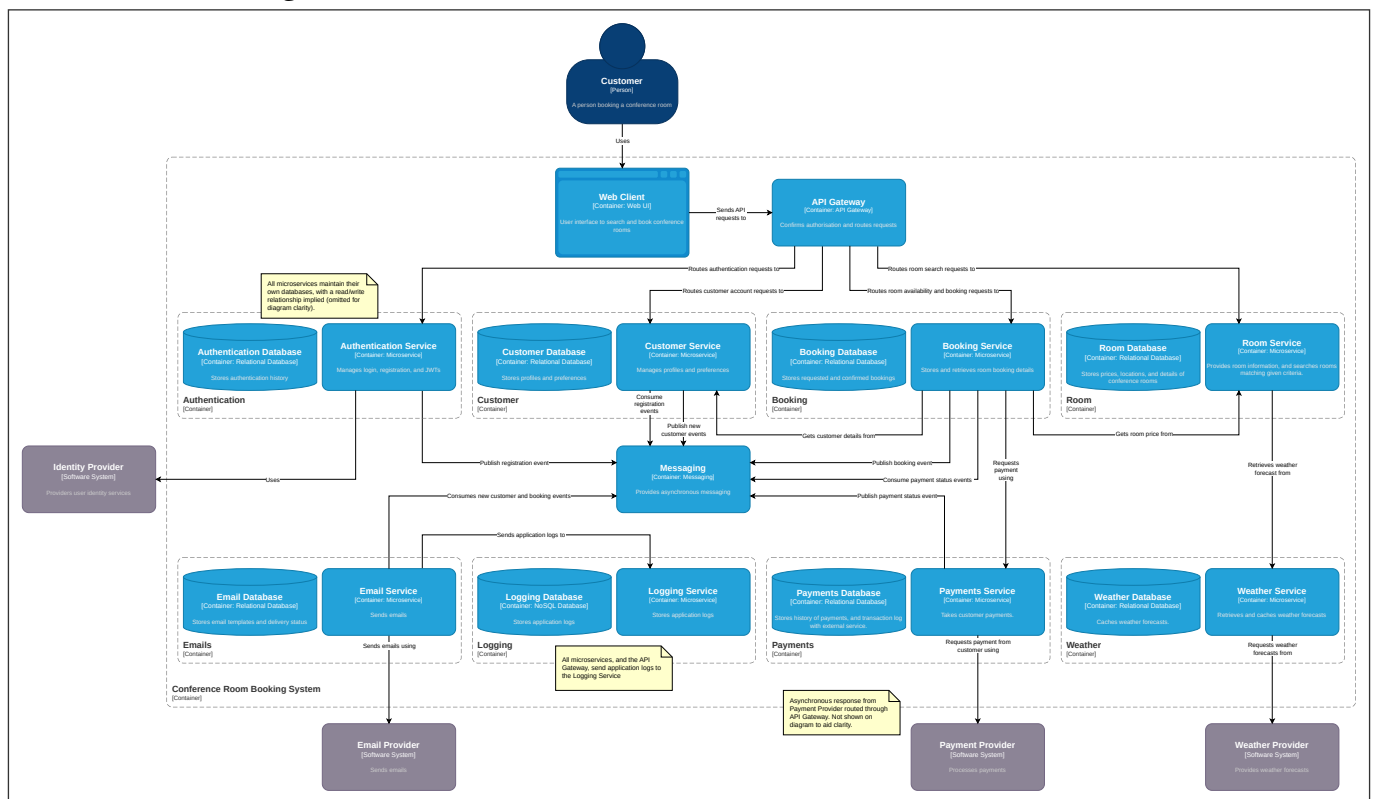


Figure 1: Technology Agnostic Container Diagram (C4 Level 2)

The proposed solution depends on four third-party services to provide authentication, online payments, email, and weather forecasts, which will all be mocked. A technology agnostic C4 Level 2 diagram is shown in Figure 1. For assistance in understanding the system's context, see Figure 2 of Appendix B.

Each component is described in Table 11 of Appendix C. Asynchronous events, published to and consumed from the *Message Queue*, are listed in Table 12 of Appendix D. C4 dynamic diagrams are provided in Figure 3 to Figure 6 of Appendix E.

AWS [1] has been chosen as the cloud computing platform due to its maturity, flexible pricing structure, excellent documentation, and the availability of online support. It is also the platform that I am most familiar with.

3. Technology Stack

A comparison of each technology required in our system is provided below. The C4 Level 2 diagram has been updated to reflect these decisions, shown in Figure 7 of Appendix F. Table 13 of Appendix G provides a matrix table of the components and the technologies that they use. Where available, AWS technology has been selected, to ensure tight integration. Serverless is the preferred option, reducing maintenance and costs.

Relational databases have been chosen for most of our services. A well-structured, ACID compliant, relational database ensures data integrity, and our services can take advantage of the strength and flexibility of SQL's querying power. Multiple instances of the same service can operate on the same data concurrently, without risk of interfering with each other.

However, our *Logging Service* is required to quickly store unstructured, or semi-structured, data from multiple services. Furthermore, complex queries will not be required for this service. Data will be short-lived, as old logs will be removed, and so deletion should be easy. To meet these requirements, a **NoSQL** database will be chosen for the *Logging Service*.

React.js [2]	Vue.js [3]
- Library with dependence on "community standard" libraries for common solutions - Uses JSX for HTML generation - Very good documentation	- Framework bundled with official libraries for common solutions - Enforced project structure - HTML templates, with conditional rendering, such as <code>v-if</code> and <code>v-for</code> - Excellent documentation with links to quality videos
My only exposure to front-end development is PHP and raw HTML. After working through some online tutorials I found the approach that Vue.js takes to be more logical. I found the Vue.js documentation easier to follow, and it provides a "Playground" to test examples. It also provides links to free videos on sites such as Scrimba and Vue School. Piastrou [4] found Vue to provide "... a balanced approach with competitive performance, lower memory usage, and effective scalability ..." while React "... demonstrated efficient performance ..., though it has a tendency to result in performance bottlenecks if it uses very complex state management unless appropriately optimized." Because I have found it easier to learn, and with its "low memory usage, and effective scalability", I have opted for Vue.js to develop the Web Client.	

Table 1: Web Client: Framework

HTTP API [5], [6], [7]	REST API [5], [6], [7]
- Simple forwarding - Built-in JWT authoriser - No caching	- Feature-rich forwarding - JWT authorisation using Lambda - Caching support - More expensive
Though HTTP API does not provide caching as REST API does, it is the cheaper option and, with fewer features, may be easier to set up. The built-in JWT authoriser (without the need of a Lambda) is also an advantage to help us manage authorisation. Cloud Map [8] will also be required to allow the API Gateway to discover our services.	

Table 2: API Gateway

MySQL [9]	PostgreSQL [10]
- Very widely used - Smaller feature-set - Widespread community support and online tutorials	- Less popular - Larger feature-set - Advanced data types
The majority of our services will benefit from the structured querying and data integrity provided by a relational database. I am choosing to use PostgreSQL for its advanced data types, including JSON.	

Table 3: Services: Relational Database

DocumentDB [11], [12]	DynamoDB [13], [14]
- Document model - MongoDB compatible - Access only available to services within the same VPC - Priced per second, with a 10-minute minimum	- Key-value model - Time-to-Live feature to delete old log entries - Priced per request
A NoSQL database is required for the <i>Logging Service</i> so that we may take advantage of its fast write speeds, and store semi-structured data from multiple components. However, we have no requirement to implement logging analysis, so a document model is considered unnecessary. DynamoDB is especially designed for high throughput scenarios such as logging, and scales to demand. Its TTL feature will allow old logs to be automatically deleted or (with a Lambda) archived, making it, along with its low cost, my choice of NoSQL database for the Logging Service.	

Table 4: Logging Service: NoSQL Database

Go with Gin-Gonic [15], [16]	JavaScript with Node.js [17]
• Compiled language with static typing • Small resource footprint • Fast cold-start time • Compiler caught syntax and type errors • Goroutines offer lightweight concurrency	• Interpreted language with dynamic typing • Large resource footprint • Longer cold-start time • Single-threaded, I/O bound
Go's small resource footprint, and fast cold start time, will reduce AWS costs, and allow faster deployment. While interpreted languages, such as JavaScript, can lead to faster initial development time, in larger systems this is more than offset by fewer bugs reaching production, and easier maintenance, thanks to compiler caught errors. Furthermore, according to Pike [18], one of Go's creators [19], Go was designed for scalability and fast compilation. I observed fast (near instantaneous) compilation times when experimenting with Go myself. Though not crucial to our design at present, as most blocking will be I/O bound (which Javascript's single thread model handles natively), Lumba [20] found that Goroutines are, on average, 6.25 times faster than a thread-based model. Gin-Gonic is a popular Go framework, currently with 86,966 stars on GitHub [21], ensuring plenty of community support. It aims to be fast, with low memory usage, and includes JSON request validation. Kuffel and Walter [22] used Gin-Gonic as a reference when comparing Node.js frameworks for performance in five different scenarios, and found Gin-Gonic to be significantly faster than any of the Node.js frameworks, in all scenarios except when a server error occurs (which they linked to Go's error handling method).	

Table 5: Services: Language & Framework

Simple Notification Service (SNS) [23], [24]	Simple Queue Service (SQS) [25], [26]
• Push-based pub/sub model • No persistence	• Pull-based queue • Messages stored for up to 14 days • Dead letter queue
Both options provide us with serverless solutions. The pull-based model of SQS suits our system better, as each event type is only consumed by one service, and the message persistence guarantees consumption. The dead letter queue will help us identify and resolve system errors.	

Table 6: Messaging

Elastic Container Service (ECS) [27], [28]	Elastic Kubernetes Service (EKS) [29], [30]
• Simplified container management • No additional fees	• Open source industry standard • Cost per hour
EKS is an open source industry standard, which would reduce our dependence on the AWS ecosystem while retaining tight integration. It also allows us to employ Kubernetes software. However, it has an hourly charge, whereas ECS is free. ECS is also designed to be simple and to integrate well with other AWS products. Therefore I have opted to use ECS. This will require the use of Docker [31] to create the images for ECS to run.	

Table 7: Container Deployment

Elastic Compute Cloud (EC2) [32], [33]	Fargate [34], [35]
• Full control over resource • Requires manual maintenance (patching etc.) • Cost per hour	• Fully managed serverless solution • Charged according to usage — no cost per hour • Automatic scaling • Can scale to zero
We have no requirement to manually configure our compute, and so with its reduced price, reduced maintenance requirement, and automatic scaling, Fargate is the preferred solution.	

Table 8: Container Compute

Service Connect [36]	VPC Lattice [37]
• Supports HTTP/HTTPS/TCP • Uses friendly DNS names • No additional charge	• Supports HTTP/HTTPS/TCP/gRPC • Allows communication across VPCs and accounts • Charged by hour and resources consumed
Service Connect is the simpler solution provided by AWS, and has no additional charge beyond the resources that it consumes in the container task. While Lattice allows communication between VPCs and accounts, we have no need for this feature. Therefore the system services will communicate using Service Connect, over a REST interface, utilising the Cloud Map already used by our API Gateway.	

Table 9: Service Communication

4. CI/CD Pipeline

GitHub [38] will be used for source code version control. The pipeline will use GitHub Actions [39], [40], and be triggered by an update to the main branch. A pipeline flowchart is provided in Figure 8 of Appendix H.

Infrastructure as Code (IaC)

Provisioning of our compute, running tests, and deploying to live, will be managed through an IaC. Terraform [41] and AWS CloudFormation [42] were considered. The former was chosen due to its ease of use and the agnostic flexibility that it offers both cloud and non-cloud platforms. Terraform will use OIDC credentials to connect to our AWS account, and use S3 [43], [44] to store state files.

Registry

GitHub Packages [45] and AWS Elastic Container Registry [46], [47] were considered for the container registry. Though storing our images on the same platform as our source code was considered, it was decided that using ECR would prevent introducing additional authentication complexity.

Test Suites

UI: Playwright [48] for its popularity and GitHub Action support [49].

Unit & Integration: Pytest [50] for its wide community support and extensive list of plugins [51].

Load: Locust [52] can be run in headless mode, while periodically saving results to CSV files. It uses a Python code architecture.

Tests will be run in an AWS test environment, created in a dedicated test VPC. A notification will be sent to a review team, who will need to manually approve the code going live.

Docker Compose [53] will be used for local development and testing, to ensure consistency with the live environment.

Deployment

A Blue/Green Deployment [54] strategy will be used to deploy the *previously built images* to the live environment, utilising AWS Code Deploy [55], [56]. **Pappula** [57] observed that such a method allows extensive testing of the new environment, and quick rollback if necessary, at the cost of increased resource costs during deployment. He found that “*the core metrics ... improved substantially, with no downtimes and reduced error rates, by 100% and 80.95%, respectively, and latency in updating an application was reduced by 20%.*” while **Vangala** [58] found “[*e*]xperimental results indicated Blue-Green Deployment achieved a deployment success rate of 98.8% and maintained a recovery time of under 1.5 minutes.”

Appendix A: Temperature Price Adjustments

The system requirements state that conference room prices are adjusted by the temperature forecast for the day of the booking, as shown in Table 10, where T = temperature.

$0^{\circ}C \leq T < 2^{\circ}C$	No additional charge
$2^{\circ}C \leq T < 5^{\circ}C$	10%
$5^{\circ}C \leq T < 10^{\circ}C$	20%
$10^{\circ}C \leq T < 20^{\circ}C$	30%
$20^{\circ}C \leq T$	50%

Table 10: Price Adjustments

Appendix B: C4 System Context Diagram

Figure 2 shows the context of the booking system. The proposed solution depends on four third-party services to provide authentication, online payments, email, and weather forecasts.

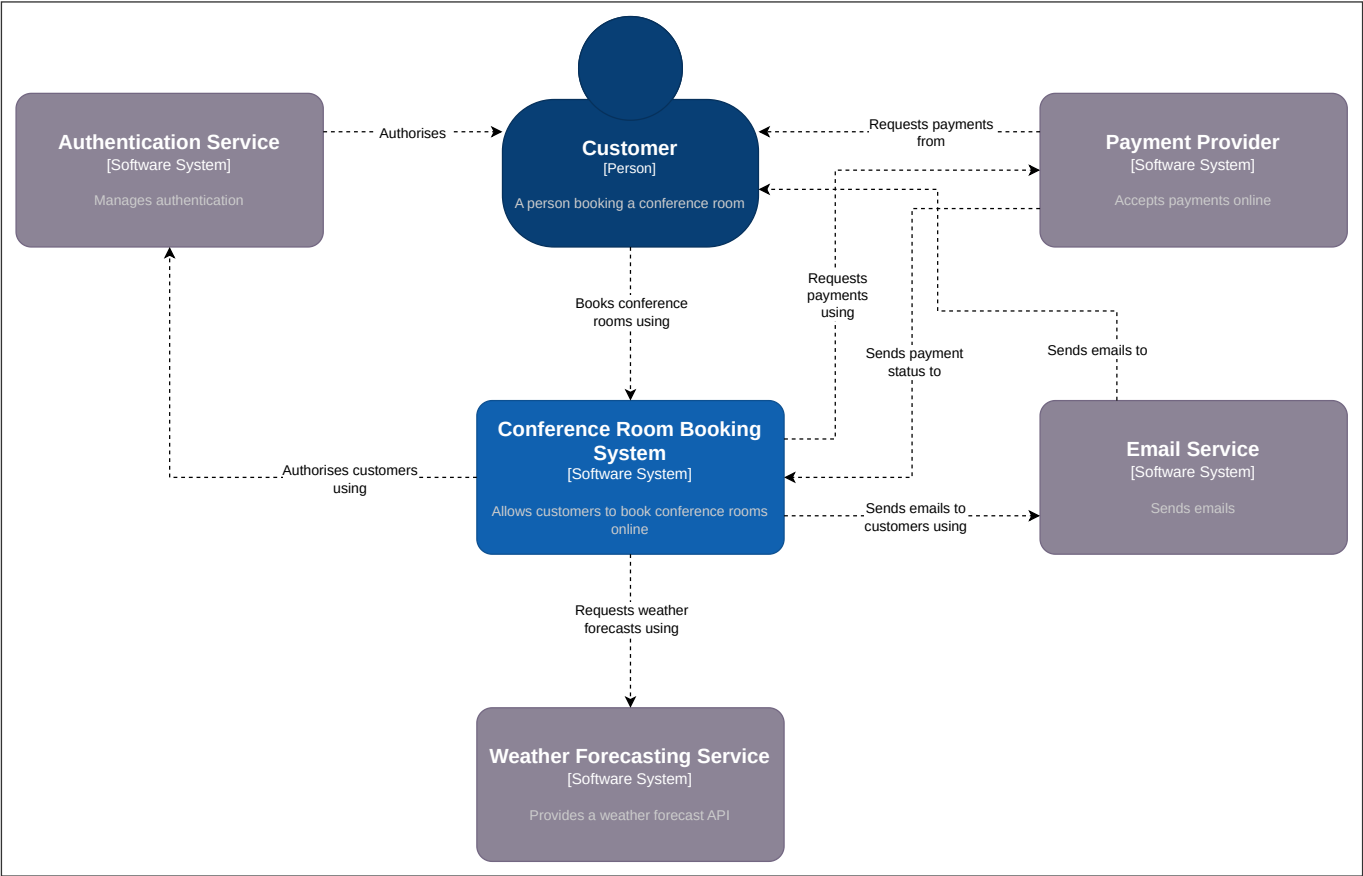


Figure 2: System Context Diagram (C4 Level 1)

Appendix C: System Components

Table 11 lists the eleven main components proposed for the system, and a brief description of each.

Web Client (presentation)	User interface to search and book conference rooms. All requests are routed through the <i>API Gateway</i> .
API Gateway (presentation)	Routes requests from the <i>Web Client</i> to the appropriate service, and confirms <i>Web Client</i> authorisation.
Authentication (business)	Provides management of login and registration security details. Creates new credentials and publishes a registration event to the <i>Message Queue</i> .
Customer (business)	Maintains customer account profiles. Consumes registration events from the <i>Message Queue</i> to create new profiles, before publishing a new customer event. Provides customer details to the <i>API Gateway</i> and other microservices.
Booking (business)	Stores confirmed and requested bookings. Allows the <i>API Gateway</i> to request a conference room booking, and to check existing bookings. Consumes payment events, and publishes booking events.
Room (business)	Maintains descriptions, photos, prices, and capacities, of the conference rooms. Uses the <i>Weather Service</i> to retrieve weather forecasts, so that a room price may be adjusted according to the expected temperature. Provides the <i>API Gateway</i> with a conference room search facility. The <i>Web Client</i> must get the booking status from the <i>Booking Service</i> .
Email (support)	Consumes new customer and booking events from the <i>Message Queue</i> to email customers. Stores email templates in its database.
Logging (support)	Provides a logging facility to all microservices. Each microservice, and the <i>API Gateway</i> , has a “Sends application logs to” relationship with the <i>Logging Service</i> (not shown in the diagram to maintain clarity).
Payment (support)	Processes customer payments for the <i>Booking Service</i> , using the external <i>Payment Provider</i> . Returns the result asynchronously via the <i>Message Queue</i> by publishing a payment status event.
Weather (support)	Provides temperature information by location and date. Uses the external <i>Weather Provider</i> to retrieve forecasts at regular intervals, and caches the response locally to allow synchronous responses to the <i>Rooms Service</i> .
Message Queue (support)	Provides asynchronous communication functionality between services. Associated data is stored within the event.

Table 11: C4 System Components

Appendix D: Message Queue Events

The system publishes and consumes four events to the *Message Queue*, allowing asynchronous processing, as shown in table Table 12.

Web Client (presentation)	User interface to search and book conference rooms. All requests are routed through the <i>API Gateway</i> .
Registration	Published when new authentication credentials are created. Consumed by the <i>Customer Service</i> to create a new customer profile.
New Customer	Published by the <i>Customer Service</i> after a profile is created. Consumed by the <i>Email Service</i> to send a welcome message.
Payment Status	Published by the <i>Payment Service</i> when a payment transaction completes, regardless of success or failure. Consumed by the <i>Booking Service</i> to confirm the booking in its database, or to remove it.
Booking	Published by the <i>Booking Service</i> when a room booking is paid for and confirmed. Consumed by the <i>Email Service</i> to send a confirmation and receipt to the customer.

Table 12: Message Queue Events

Appendix E: C4 Dynamic Diagrams

Figure 3 to Figure 6 are C4 dynamic diagrams, illustrating how the system components collaborate for the four main runtime requests in our system. Note that to maintain clarity, communication with the *Logging Service* is not shown.

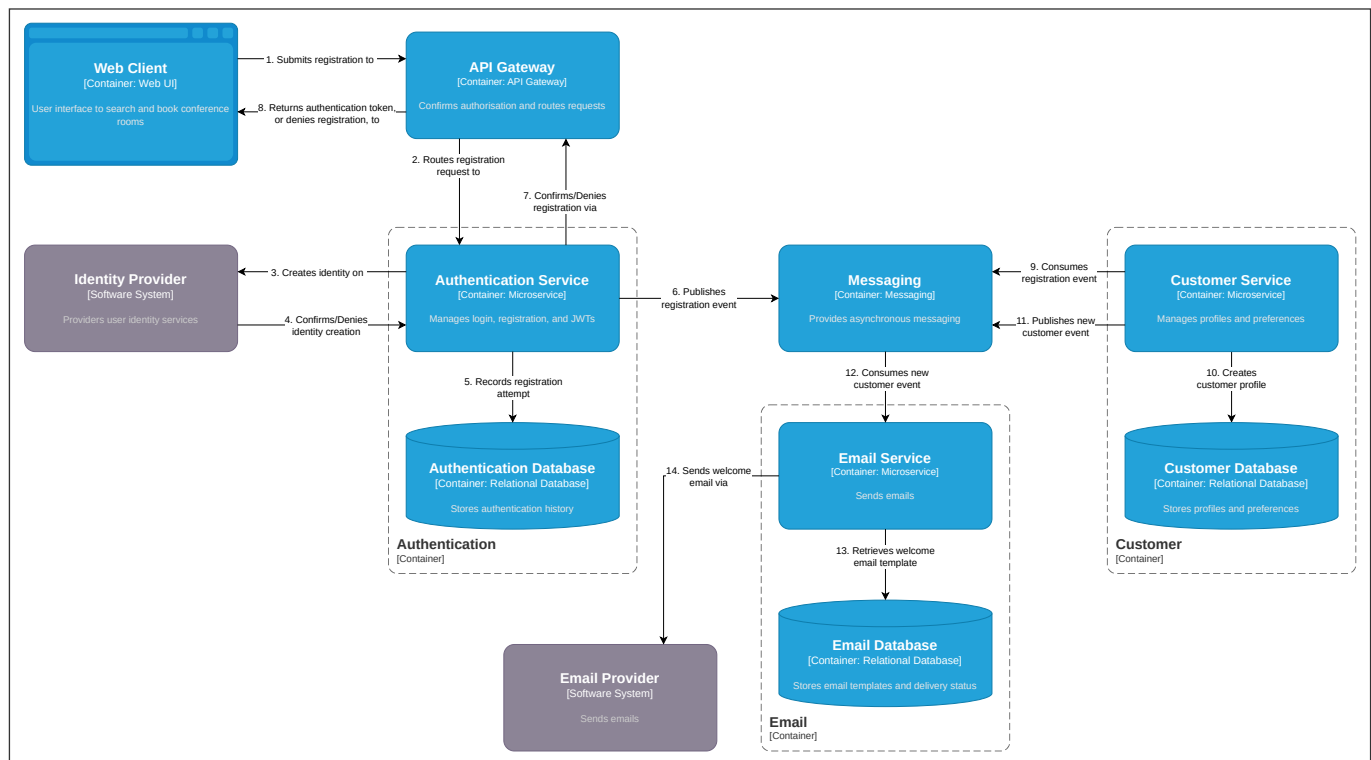


Figure 3: New Customer Registration

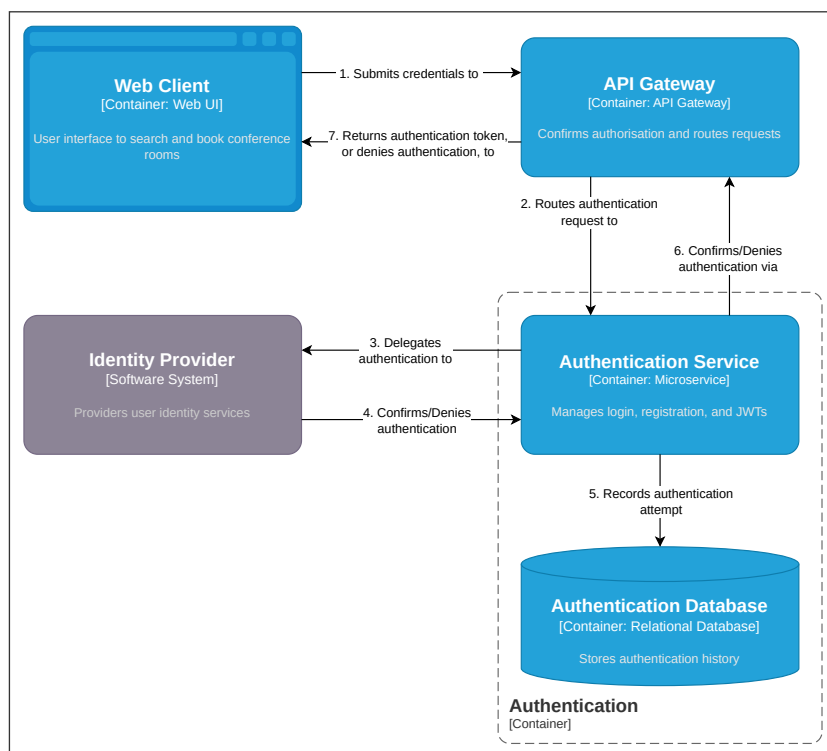


Figure 4: Customer Authentication

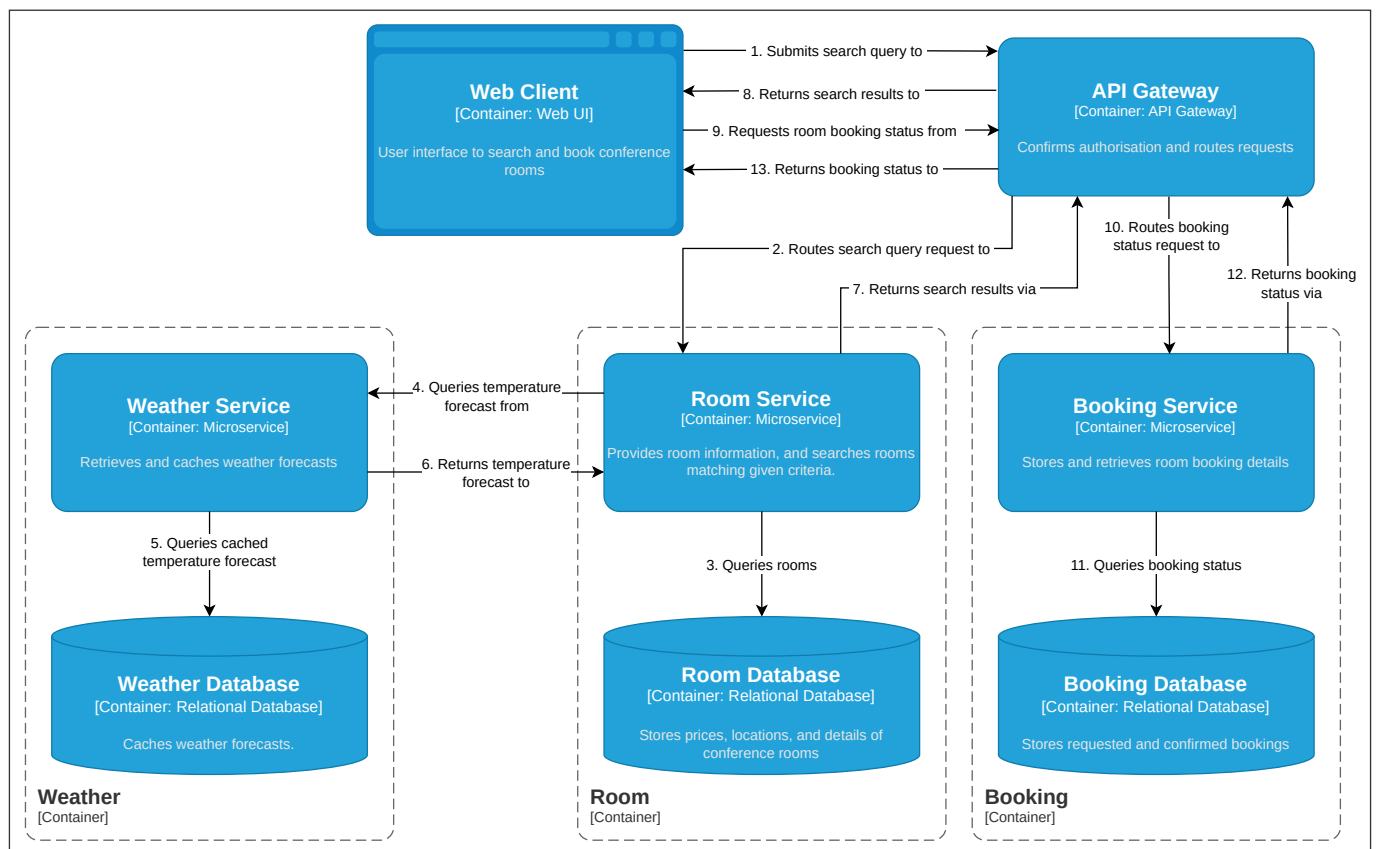


Figure 5: Room Search

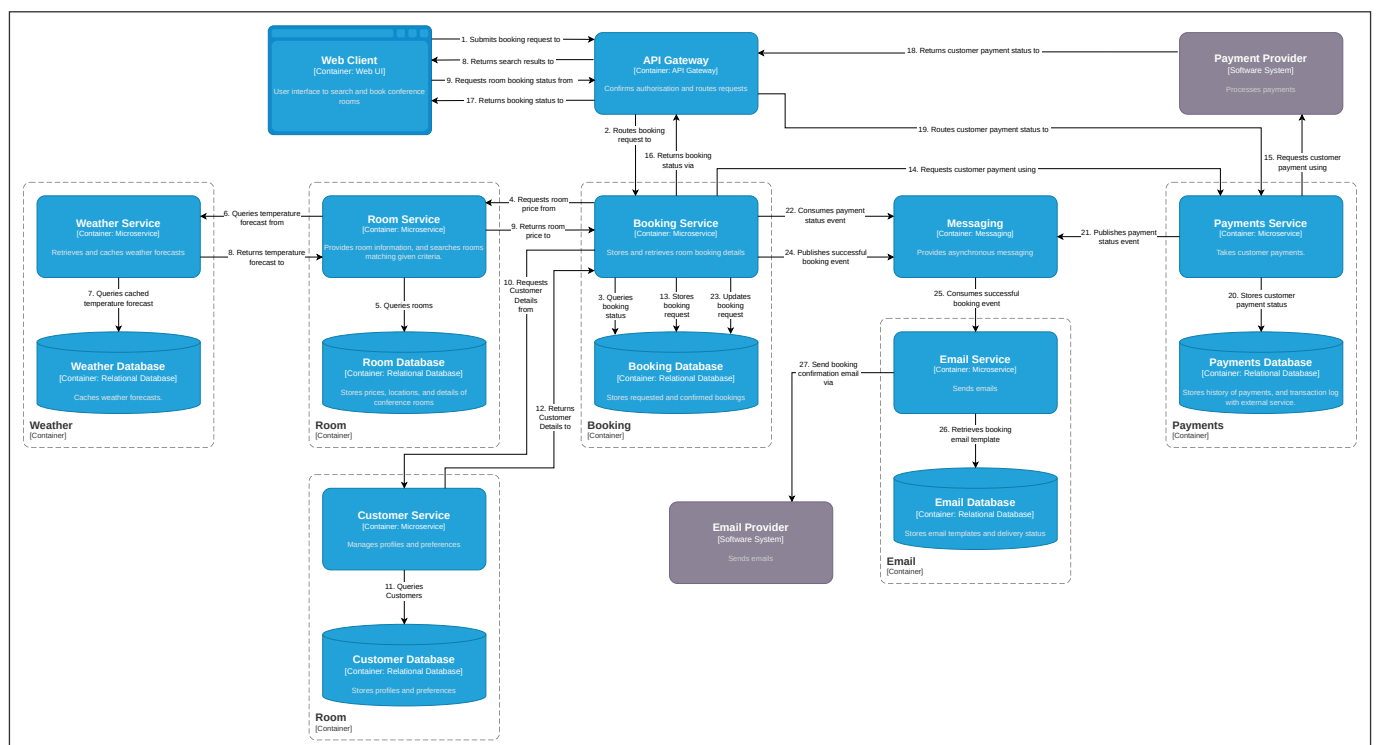


Figure 6: Successful Room Booking Request

Appendix F: Technology Specific Container Diagram

Figure 7 extends the Level 2 Agnostic diagram (Figure 1) with the technology stack identified in Section 3.

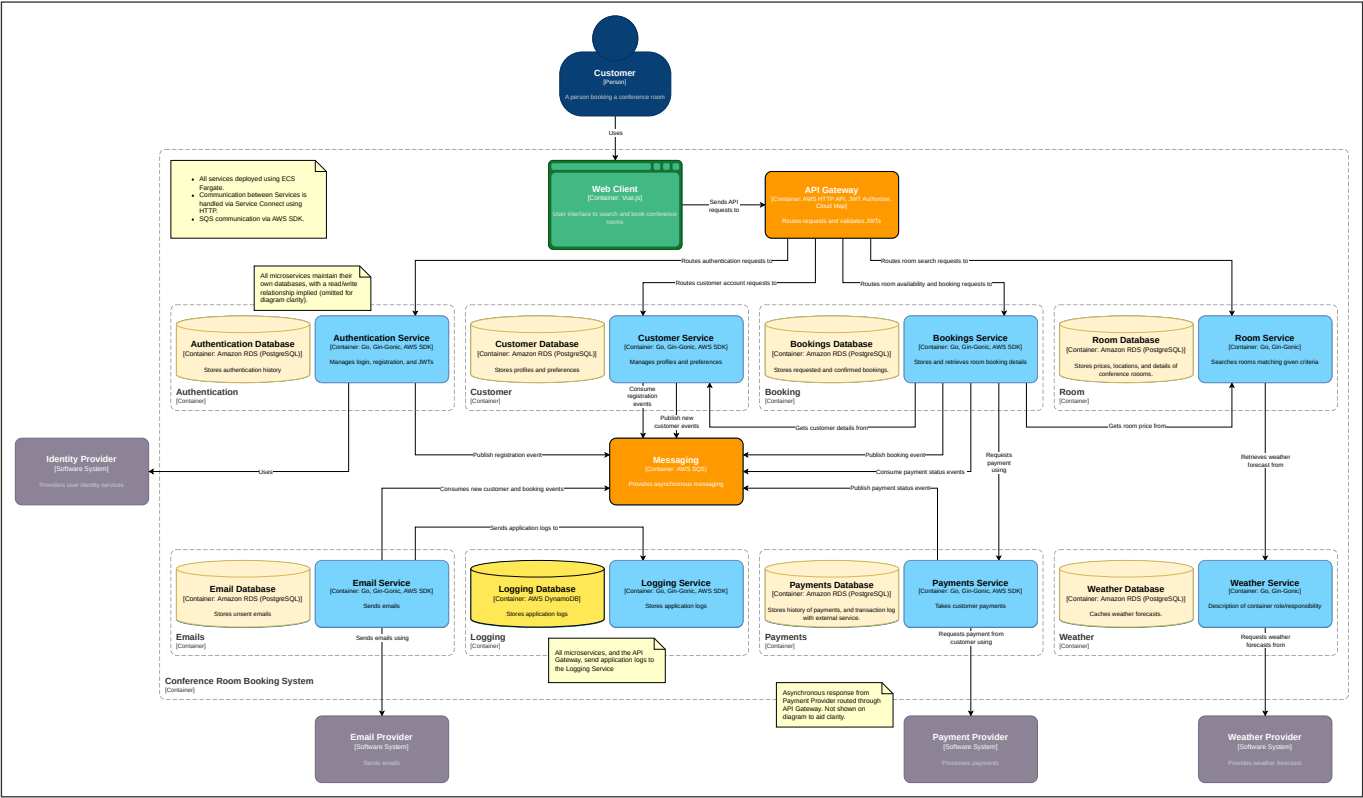


Figure 7: Technology Specific Container Diagram (C4 Level 2)

Appendix G: Technology Matrix Table

Table 13 details the relationship between our components identified in Section 2, and the technologies identified in Section 3. Note that the *Messaging* component is located in the top row, where it becomes more useful in this context. A relationship with Gin-Gonic implies a relationship with Go. “S. Connect” is an abbreviation of “Service Connect”.

	Vue.js	HTTP API	JWT Authorizer	Cloud Map	Gin-Gonic	PostgreSQL	DynamoDB	SQS (Messaging)	ECS	Fargate	S. Connect
Web Client	•										
API Gateway		•	•	•							
Authentication					•	•		•	•	•	•
Customer					•	•		•	•	•	•
Bookings					•	•		•	•	•	•
Room					•	•			•	•	•
Email					•	•		•	•	•	•
Logging					•		•		•	•	
Payments					•	•		•	•	•	•
Weather					•	•			•	•	•

Table 13: Technology Matrix

Appendix H: CI/CD Pipeline Flowchart

Figure 8 presents a flowchart for our CI/CD pipeline, as described in Section 4.

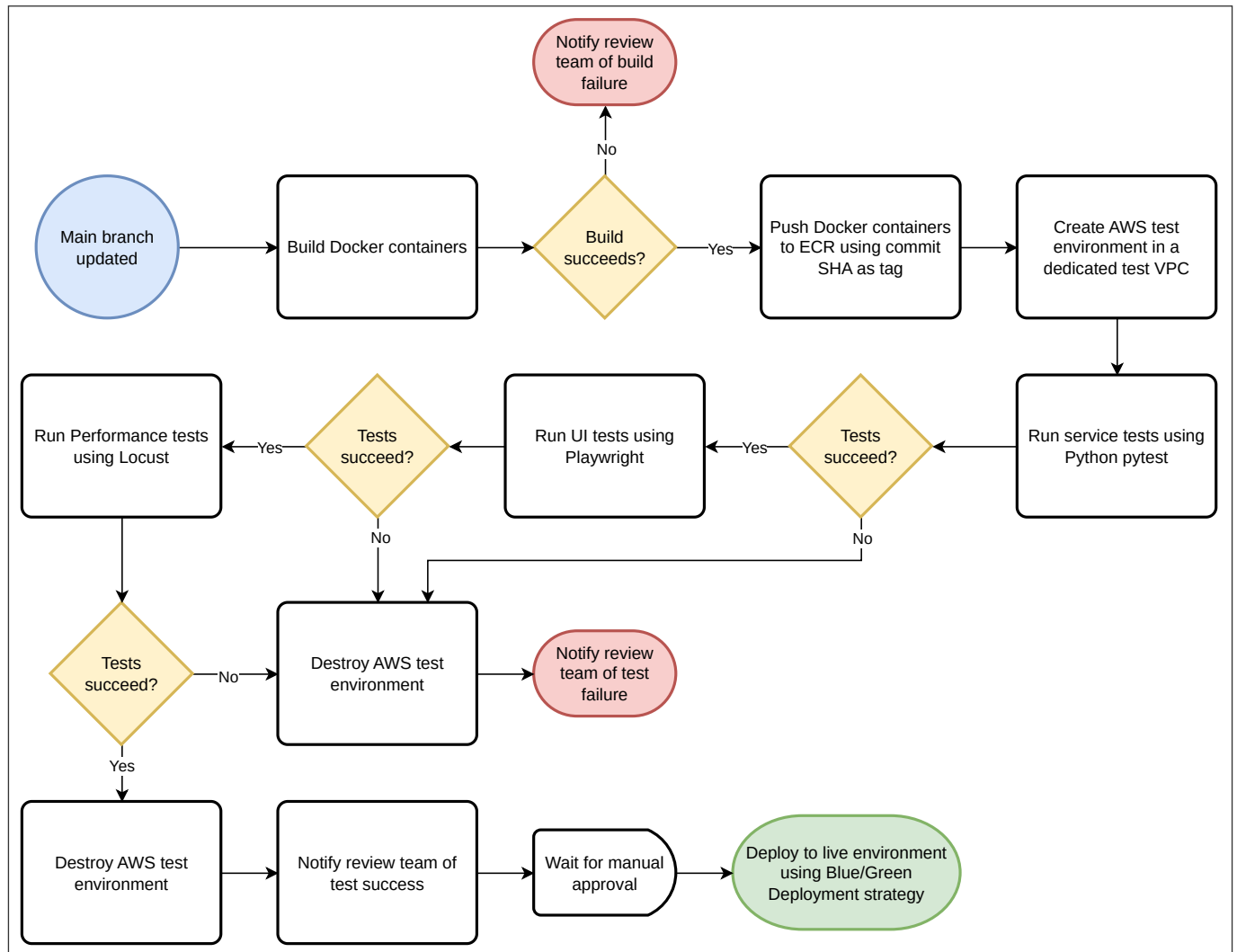


Figure 8: CI/CD Pipeline Flowchart

References

- [1] "Cloud Computing Services - Amazon Web Services (AWS)." Accessed: Nov. 15, 2025. [Online]. Available: <https://aws.amazon.com/>
- [2] "React." Accessed: Nov. 13, 2025. [Online]. Available: <https://react.dev/>
- [3] "Vue.js." Accessed: Nov. 12, 2025. [Online]. Available: <https://vuejs.org/>
- [4] M. Piastou, "Comprehensive Performance and Scalability Assessment of Front-End Frameworks: React, Angular, and Vue.js," *World Journal of Advanced Engineering Technology and Sciences*, vol. 9, no. 2, pp. 366–376, 2023, doi: [10.30574/wjaets.2023.9.2.0153](https://doi.org/10.30574/wjaets.2023.9.2.0153).
- [5] "API Management - Amazon API Gateway - AWS." Accessed: Nov. 14, 2025. [Online]. Available: <https://aws.amazon.com/api-gateway/>
- [6] "Amazon API Gateway FAQs." Accessed: Nov. 14, 2025. [Online]. Available: <https://aws.amazon.com/api-gateway/faqs/>
- [7] "Choose between REST APIs and HTTP APIs - Amazon API Gateway." Accessed: Nov. 13, 2025. [Online]. Available: <https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-vs-rest.html>
- [8] "IT Asset Discovery - AWS Cloud Map - AWS." Accessed: Nov. 15, 2025. [Online]. Available: <https://aws.amazon.com/cloud-map/>
- [9] "MySQL." Accessed: Nov. 13, 2025. [Online]. Available: <https://www.mysql.com/>
- [10] "PostgreSQL." Accessed: Nov. 13, 2025. [Online]. Available: <https://www.postgresql.org/>
- [11] "Amazon DocumentDB- Serverless, Fully Managed, MongoDB API-compatible Document Database." Accessed: Nov. 13, 2025. [Online]. Available: <https://aws.amazon.com/documentdb/>
- [12] "What Is Amazon DocumentDB (with MongoDB Compatibility) - Amazon DocumentDB." Accessed: Nov. 14, 2025. [Online]. Available: <https://docs.aws.amazon.com/documentdb/latest/developerguide/what-is.html>
- [13] "Amazon DynamoDB." Accessed: Nov. 13, 2025. [Online]. Available: <https://aws.amazon.com/dynamodb/>
- [14] "What Is Amazon DynamoDB? - Amazon DynamoDB." Accessed: Nov. 14, 2025. [Online]. Available: <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html>
- [15] "The Go Programming Language." Accessed: Nov. 13, 2025. [Online]. Available: <https://go.dev/>
- [16] "Gin Web Framework." Accessed: Oct. 08, 2025. [Online]. Available: <https://gin-gonic.com/>
- [17] "Node.js — Run JavaScript Everywhere." Accessed: Nov. 13, 2025. [Online]. Available: <https://nodejs.org/en>
- [18] "Go at Google: Language Design in the Service of Software Engineering - The Go Programming Language." Accessed: Nov. 13, 2025. [Online]. Available: <https://go.dev/talks/2012/splash.article>
- [19] R. Cox, R. Griesemer, R. Pike, I. L. Taylor, and K. Thompson, "The Go Programming Language and Environment," *Commun. ACM*, vol. 65, no. 5, pp. 70–78, Apr. 2022, doi: [10.1145/3488716](https://doi.org/10.1145/3488716).
- [20] S. Lumba, "Enhancing Microservice Performance: A Comparative Analysis of Thread & Fiber Model," *mathesis*, 2024. Accessed: Nov. 13, 2025. [Online]. Available: <https://norma.ncirl.ie/7088/>
- [21] "Gin-Gonic/Gin." Accessed: Nov. 13, 2025. [Online]. Available: <https://github.com/gin-gonic/gin>
- [22] P. Kuffel and B. Walter, "Performance of Node.js Backend Application Frameworks. An Empirical Evaluation," *Proceedings of the International Conference on Information Systems Development (ISD)*, Sept. 2024, [Online]. Available: <https://aisel.aisnet.org/isd2014/proceedings2024/managingdevops/7>
- [23] "Push Notification Service - Amazon Simple Notification Service - AWS." Accessed: Nov. 14, 2025. [Online]. Available: <https://aws.amazon.com/sns/>
- [24] "What Is Amazon SNS? - Amazon Simple Notification Service." Accessed: Nov. 14, 2025. [Online]. Available: <https://docs.aws.amazon.com/sns/latest/dg/welcome.html>
- [25] "Message Queuing Service - Amazon Simple Queue Service - AWS." Accessed: Nov. 14, 2025. [Online]. Available: <https://aws.amazon.com/sqs/>
- [26] "What Is Amazon Simple Queue Service? - Amazon Simple Queue Service." Accessed: Nov. 14, 2025. [Online]. Available: <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/welcome.html>
- [27] "Amazon ECS." Accessed: Nov. 14, 2025. [Online]. Available: <https://aws.amazon.com/ecs/>
- [28] "Fully Managed Container Orchestration – Amazon Elastic Container Service (Amazon ECS) FAQs – AWS." Accessed: Nov. 14, 2025. [Online]. Available: <https://aws.amazon.com/ecs/faqs/>
- [29] "Managed Kubernetes - Amazon Elastic Kubernetes Service (EKS) - AWS." Accessed: Nov. 14, 2025. [Online]. Available: <https://aws.amazon.com/eks/>

- [30] "Amazon EKS - FAQs." Accessed: Nov. 14, 2025. [Online]. Available: <https://aws.amazon.com/eks/faqs/>
- [31] "Docker: Accelerated Container Application Development." Accessed: Nov. 15, 2025. [Online]. Available: <https://www.docker.com/>
- [32] "Amazon EC2 - Cloud Compute Capacity - AWS." Accessed: Nov. 14, 2025. [Online]. Available: <https://aws.amazon.com/ec2/>
- [33] "Amazon EC2 FAQs." Accessed: Nov. 14, 2025. [Online]. Available: <https://aws.amazon.com/ec2/faqs/>
- [34] "Serverless Compute - AWS Fargate - AWS." Accessed: Nov. 14, 2025. [Online]. Available: <https://aws.amazon.com/fargate/>
- [35] "Serverless Compute Engine – AWS Fargate FAQs – Amazon Web Services." Accessed: Nov. 14, 2025. [Online]. Available: <https://aws.amazon.com/fargate/faqs/?nc=sn&loc=5.html>
- [36] "Use Service Connect to Connect Amazon ECS Services with Short Names - Amazon Elastic Container Service." Accessed: Nov. 14, 2025. [Online]. Available: <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/service-connect.html>
- [37] "Use Amazon VPC Lattice to Connect, Observe, and Secure Your Amazon ECS Services - Amazon Elastic Container Service." Accessed: Nov. 14, 2025. [Online]. Available: <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/ecs-vpc-lattice.html>
- [38] "GitHub · Change Is Constant. GitHub Keeps You Ahead.." Accessed: Nov. 15, 2025. [Online]. Available: <https://github.com/>
- [39] "GitHub Actions." Accessed: Nov. 15, 2025. [Online]. Available: <https://github.com/features/actions>
- [40] "GitHub Actions Documentation." Accessed: Nov. 15, 2025. [Online]. Available: <https://docs-internal.github.com/en/actions>
- [41] "Terraform | HashiCorp Developer." Accessed: Nov. 15, 2025. [Online]. Available: <https://developer.hashicorp.com/terraform>
- [42] "What Is AWS CloudFormation? - AWS CloudFormation." Accessed: Nov. 16, 2025. [Online]. Available: <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/Welcome.html>
- [43] "Amazon S3 - Cloud Object Storage - AWS." Accessed: Nov. 16, 2025. [Online]. Available: <https://aws.amazon.com/s3/>
- [44] "S3 FAQs All." Accessed: Nov. 16, 2025. [Online]. Available: <https://aws.amazon.com/s3/faqs/>
- [45] "GitHub Packages Documentation." Accessed: Nov. 16, 2025. [Online]. Available: <https://docs-internal.github.com/en/packages>
- [46] "Container Registry - Amazon Elastic Container Registry (Amazon ECR) - AWS." Accessed: Nov. 16, 2025. [Online]. Available: <https://aws.amazon.com/ecr/>
- [47] "What Is Amazon Elastic Container Registry? - Amazon ECR." Accessed: Nov. 16, 2025. [Online]. Available: <https://docs.aws.amazon.com/AmazonECR/latest/userguide/what-is-ecr.html>
- [48] "Fast and Reliable End-to-End Testing for Modern Web Apps | Playwright." Accessed: Nov. 15, 2025. [Online]. Available: <https://playwright.dev/>
- [49] "Setting up CI | Playwright." Accessed: Nov. 16, 2025. [Online]. Available: <https://playwright.dev/docs/ci-intro>
- [50] "Pytest Documentation." Accessed: Nov. 15, 2025. [Online]. Available: <https://docs.pytest.org/en/stable/>
- [51] "Pytest Plugin List - Pytest Documentation." Accessed: Nov. 16, 2025. [Online]. Available: https://docs.pytest.org/en/stable/reference/plugin_list.html
- [52] "Locust.Io." Accessed: Nov. 15, 2025. [Online]. Available: <https://locust.io/>
- [53] "Docker Compose." Accessed: Nov. 16, 2025. [Online]. Available: <https://docs.docker.com/compose/>
- [54] "Blue/Green Deployments on AWS - Blue/Green Deployments on AWS." Accessed: Nov. 16, 2025. [Online]. Available: <https://docs.aws.amazon.com/whitepapers/latest/blue-green-deployments/welcome.html>
- [55] "Automated Code Deployment - AWS CodeDeploy - AWS." Accessed: Nov. 16, 2025. [Online]. Available: <https://aws.amazon.com/codedeploy/>
- [56] "What Is CodeDeploy? - AWS CodeDeploy." Accessed: Nov. 16, 2025. [Online]. Available: <https://docs.aws.amazon.com/codedeploy/latest/userguide/welcome.html>
- [57] K. K. Pappula, "Containerized Zero-Downtime Deployments in Full-Stack Systems," *International Journal of AI, BigData, Computational and Management Studies*, vol. 3, no. 4, pp. 60–69, Dec. 2022, doi: [10.63282/3050-9416.IJAIABDCMS-V3I4P107](https://doi.org/10.63282/3050-9416.IJAIABDCMS-V3I4P107).
- [58] V. Vangala, "Blue-Green and Canary Deployments in DevOps: A Comparative Study," *International Journal of Engineering Sciences & Research Technology*, vol. 15, pp. 1047–1063, Apr. 2020, doi: [10.5281/zenodo.15483641](https://doi.org/10.5281/zenodo.15483641).